Самаризация доклада: «Последние несколько месяцев полностью разрушили мою уверенность в себе»

Суть

Автор — опытный разработчик и мейнтейнер Open Source — фиксирует слом привычной парадигмы разработки под влиянием массового прихода LLM-агентов. Ключевой фактор — **объём кода**: за год число коммитов на GitHub выросло в 15 раз при том же числе программистов. Причина — роботы, а не люди. Этот «вал кода» снёс прежние принципы и заставил сформулировать **семь новых**.

Семь новых принципов

1. Код не должен быть читаемым — он должен быть минимальным

- Комментарии больше не нужны: люди код не читают, а агенты от комментариев только путаются (особенно если комментарий расходится с кодом).
- Практика: полный запрет комментариев в build-скриптах, Haskell, JavaScript; Java — на пути к запрету.
- Приоритет: минимализм, точность, «делать только необходимое».

2. Понятие legacy-кода упраздняется

- «Legacy» — это код, который ещё не попал в руки Claude Code.
- Кейс: Java-статический анализатор 10–12-летней давности с 400+ тикетами в бэклоге восстановлен за неделю работы с агентом (без ручного кода, только «next, next, next»). Аналогичный кейс из статьи: 3 недели и несколько тысяч долларов вместо 1,5 лет.
- Старые репозитории не выбрасывают — их «оживляют».

3. Ручной ревью и триаж — теперь вред, а не добродетель

- У автора ~100 баг-репортов в день на репозиторий (~1,5k звёзд), суммарно ~1000 в день на несколько проектов — физически неподъёмно.
- Решение: агент каждое утро автоматически триажит репорты и мержит PR. За ночь в кодбазу попадает 40–50 PR без участия человека.
- Исключение: сложные/радикальные изменения агент оставляет на ручной ревью с тегом автора (несколько штук в день).
- Вывод: человек как ревьюер уступает машине и тормозит проект.

4. Вежливость с контрибьюторами больше не нужна

- На другой стороне PR — не программист, а «курьер», нажавший кнопку. Ему безразличны тон и эмоции.
- Можно позволить максимальную строгость: закрывать нежелательные PR/репорты без объяснений и переживаний.

5. Zero tolerance через автоматические ограничители

- Пример: CI автоматически отклоняет PR >200 строк (added+removed) — красный флаг, без возможности апелляции.
- Логика: робот увидит красный статус и сам разобьёт изменение на части. Никто не обидится.
- Принцип: всё, что считается низким качеством, закладывается в автоматические проверки → рост качества.

6. Вместо чтения диффов — «impact» и визуальные артефакты

- Код (в т.ч. сгенерированный агентами) человек больше не читает; Роберт Мартин публично подтвердил, что тоже не читает исходники.
- Решение: изменения должны приводить к изменению **визуально читаемых артефактов** — например, YAML-фикстур (вход-код → выход-XML). Автор визуально принимает/отклоняет изменения, не вникая в Java-код.
- CI должен выдавать **impact-отчёт**: не «что изменили», а «к чему привело» — метрики производительности, объёма, качества, изменения артефактов.
- Время тратится не на ручной ревью, а на настройку инструментов контроля impact.

7. Стабильность master-ветки — идея прошлого; приоритет — скорость

- Классический CI (preflight build, «шапка позора» за сломанную сборку) мёртв: 100 PR × 10 минут сборки = неподъёмно, ресурсов GitHub Actions не хватает, каждый мерж инвалидирует остальные PR.
- Принятая реальность: master почти постоянно сломан (падает ~2 раза в день). Как только краснеет — сразу запускается Claude Code, чинит, отправляет PR, его принимают, разработка продолжается.
- Вывод: жертвуем стабильностью ради скорости; иначе не успеть за объёмом.

Ключевые тезисы

Было	Стало
Код должен быть читаемым, с комментариями	Код минимальный, без комментариев
Legacy-код выбрасывают/переписывают	Репозитории «оживляют» агентами
Ревью и триаж — ответственность мейнтейнера	Автоматический триаж и мерж агентами
Вежливость с контрибьюторами	Максимальная строгость, роботу всё равно
Обсуждение размера PR	Жёсткий CI-лимит (200 строк)
Чтение диффов	Impact-отчёты и визуальные артефакты (YAML)
Стабильный master	Быстрый master, починка агентом на лету

Главная мысль: человек перестаёт быть бутылочным горлышком, передавая контроль агентам; его новая задача — настраивать автоматические ограничители и инструменты контроля impact, а не читать код и общаться с контрибьюторами.